Workshop no.3 2026-2027	Module	MTI Methods and techno. of Impl.
	Branch	Master GSI 1st year
Chapter 2: SOLID principals	ملاحظة :	

- **Objectives أهداف**: SOLID principals
- **Tools أدوات** : Python

0.1 Part Practical work

The SOLID principles are fundamental guidelines in object-oriented design that help developers create code that is maintainable, scalable, and flexible. These principles were popularized by Robert C. Martin and have become a core part of clean code practices.

- Single Responsibility Principle
- Open/Closed Principle
- Liskov Substitution Principle
- Interface Segregation Principle
- Dependency Inversion Principle

مبادئ SOLID هي إرشادات أساسية في تصميم البرمجة الكائنية تساعد المطورين في إنشاء كود مرن قابل للتوسع قابل للصيانة. شاعت هذه المبادئ بفضل روبرت سي. مارتن وصارت أساسية في ممارسات الكود النظيف.

- مبدأ المسؤولية الواحدة (الفردية)
- مبدأ مفتوح/مغلق
- مبدأ استبدال ليسكوف
- مبدأ فصل الواجهات
- (عكس الاعتمادية) مبدأ عكس التبعية

Glossary مسرد

Class	فئة/ صنف
Method	أسلوب/دالة
Interface	واجهة
Implementation	تنفيذ

1. Single Responsibility Principle (SRP)

A class should have only one reason to change.

مبدأ المسؤولية الفردية : يجب أن يكون هناك سبب واحد فقط لتغيير الفئة/الصنف.

Example:

Before SRP: The `Order` class has multiple responsibilities, like calculating the total, printing the order, and saving it to the database, which violates the SRP.

فئة `Order` لديها العديد من المسؤوليات، مثل حساب المجموع، وطباعة الطلب، وحفظه في قاعدة البيانات، وهذا ينتهك مبدأ المسؤولية الواحدة (SRP).

```python
class Order:
    def __init__(self, items):
        self.items = items

    def calculate_total(self):
        return sum(item['price'] for item in self.items)

    def print_order(self):
        print("Order details:")
        for item in self.items:
            print(f"{item['name']}: ${item['price']}")

    def save_to_db(self):
        print("Saving order to database...")
```

```
if __name__ == '__main__':
    products = [
      {"name":"fruit", "price":50},
      {"name":"vegetables", "price":100},
      {"name":"Bread", "price":20},
      {"name":"Meat", "price":1000},
     ]
    my_order = Order(products)
    total = my_order.calculate_total()
    print("Total order: ", total,"$")
    my_order.print_order()
    my_order.save_to_db()
```

After SRP:

By separating the responsibilities into different classes, each class now has a single responsibility: `Order` for managing data, `OrderPrinter` for printing, and `OrderRepository` for saving to the database.

من خلال فصل المسؤوليات إلى فئات مختلفة، أصبحت لكل فئة مسؤولية واحدة: `Order` لإدارة البيانات، `OrderPrinter` للطباعة، و`OrderRepository` للحفظ في قاعدة البيانات.

```
class Order:
    def __init__(self, items):
        self.items = items

    def calculate_total(self):
        return sum(item['price'] for item in self.items)

class OrderPrinter:
    def print_order(self, order):
        print("Order details:")
        for item in order.items:
            print(f"{item['name']}: ${item['price']}")

class OrderRepository:
    def save_to_db(self, order):
        print("Saving order to database...")
```

By separating printing and database saving, each class has a single responsibility.

عن طريق فصل الطباعة وحفظ البيانات في قاعدة البيانات، يصبح لكل صنف مسؤولية واحدة فقط.

Question: Add "main call" to test the code after SRP.

سؤال: أضف قسم main لفحص وتنفيذ الكود بعد تعديله وفق مبدأ المسؤولية الواحدة.

2. Open/Closed Principle (OCP)

Classes should be open for extension but closed for modification.

مبدأ مفتوح/مغلق: يجب أن تكون الأصناف مفتوحة للتوسيع مغلقة على التعديل.

Example:

Before OCP:

The `Discount` class applies a discount based on customer type, but if new discount types are added, the class must be modified, violating OCP.

فئة `Discount` تطبق خصمًا بناءً على نوع العميل، ولكن إذا أردنا إضافة أنواع خصومات جديدة، يجب تعديل الفئة، مما ينتهك مبدأ مفتوح/مغلق.

```python
class Discount:
    def __init__(self, customer, price):
        self.customer = customer
        self.price = price

    def apply_discount(self):
        if self.customer == 'VIP':
            return self.price * 0.8  # 20% discount
        return self.price

if __name__ == '__main__':
    price = 1000
    customer = "VIP"
    mydiscount = Discount(customer, price)
    newprice = mydiscount.apply_discount()
    print(f"Old Price:{price} for {customer}, New price {newprice}")

    price = 1000
    customer = "regular"
    # discount of 5%
    mydiscount2 = Discount(customer, price)
    newprice2 = mydiscount2.apply_discount()
    print(f"Old Price:{price} for {customer}, New price {newprice2}")
```

After OCP:

```python
class Discount:
    def __init__(self, price):
        self.price = price

    def apply_discount(self):
        return self.price

class VIPDiscount(Discount):
    def apply_discount(self):
        return self.price * 0.8  # 20% discount
```

Now, if you want to add more discount types, you can create new classes without modifying existing code.

الآن، حين ترغب في إضافة أنواع خصم جديدة، أنشئ أصنافًا جديدة دون تعديل الكود الموجود.

Questions:

1. Add a new discount for first time customer.
2. Add "main call" to test the code after OCP.

سؤال:

1. إضافة خصم جديد للزبون لأول مرة.
2. أضف قسم main لفحص وتنفيذ الكود بعد تعديله حسب مبدأ مفتوح مغلق.

3. Liskov Substitution Principle (LSP)

Subtypes should be substitutable for their base types without altering the correctness of the program.

مبدأ الاستبدال: يجب أن تكون الأنواع الفرعية قادرة على تعويض أنواعها الأساسية دون تغيير صحة البرنامج.

Example:

We have a `Rectangle` class, and we want to extend it with a `Square` class. According to geometry, a square is a special case of a rectangle.

The `Rectangle` class has `set_width()` and `set_height()` methods, while the `Square` class inherits from `Rectangle`. The problem arises because a square's width and height must always be equal, so setting the width or height independently violates the expected behavior of the `Rectangle` class.

لدينا فئة `Rectangle` (مستطيل)، ونريد توسيعها بفئة `Square` (مربع). وفقًا للهندسة، المربع حالة خاصة من المستطيل.

في فئة `Rectangle` أسلوب `set_width()` (تعيين العرض) و `set_height()` (تعيين الارتفاع)، بينما ترث فئة `Square` من `Rectangle`.

تكمن المشكلة في أن عرض المربع وارتفاعه متساويان لزومًا، وعليه فإن تعيين العرض أو الارتفاع بشكل مستقل ينتهك السلوك المتوقع لفئة `Rectangle`.

Violating LSP:

```python
# Before applying LSP (incorrect design)

class Rectangle:
    def set_width(self, width):
        self.width = width

    def set_height(self, height):
        self.height = height

    def get_area(self):
        return self.width * self.height

class Square(Rectangle):  # Square inherits from Rectangle
    def set_width(self, width):
        self.width = width
        self.height = width  # Enforces width == height

    def set_height(self, height):
        self.height = height
        self.width = height  # Enforces width == height

# Client code that expects a Rectangle
def print_area(rectangle):
    # Rectangle's area method is used
    print(f"Area: {rectangle.get_area()}")

if __name__ == '__main__':
        # Creating a rectangle and a square
        rect = Rectangle()
        rect.set_width(5)
        rect.set_height(10)

        square = Square()
        square.set_width(5)
        square.set_height(10)

        print_area(rect)   # Works fine
        print_area(square)  # This will produce incorrect results due to violation of LSP
```

In this design, when calling the `print_area()` function with a `Square`, the area calculation might not behave

as expected. The client code expects a `Rectangle`-like behavior, but the `Square` modifies the properties in an unexpected way.

عند استدعاء دالة `print_area()` مع `Square`، قد لا يعمل حساب المساحة كما هو متوقع. برنامج العميل يتوقع سلوكًا مشابهًا للمستطيل، لكن المربع يعدل الخصائص بطريقة غير متوقعة.

After LSP:

```python
# After applying LSP (correct design)

class Shape:
    def get_area(self):
        raise NotImplementedError("Must be implemented by the subclass")

class Rectangle(Shape):
    def __init__(self, width, height):
        self.width = width
        self.height = height

    def get_area(self):
        return self.width * self.height

class Square(Shape):
    def __init__(self, side_length):
        self.side_length = side_length

    def get_area(self):
        return self.side_length * self.side_length

# Client code that expects a Shape
def print_area(shape):
    print(f"Area: {shape.get_area()}")
```

In this design, we have a base class `Shape` and two subclasses: `Rectangle` and `Square`. Both `Rectangle` and `Square` correctly implement the `get_area()` method. The `Square` class does not inherit from `Rectangle` because that relationship is not suitable for the Liskov Substitution Principle. Instead, both `Rectangle` and `Square` inherit from the more general `Shape` class. Now, the `print_area()` function can accept any object of type `Shape` (either `Rectangle` or `Square`), and the behavior is consistent across both. The client code can work with both `Rectangle` and `Square` without any issues, as both correctly implement `get_area()`.

في هذا التصميم، لدينا فئة أساسية `Shape` وفئتان فرعيتان: `Rectangle` (مستطيل) و `Square` (مربع). كل من `Rectangle` و `Square` يرثان من الفئة العامة `Shape`. الآن يمكن لدالة `print_area()` قبول أي كائن من النوع `Shape` (إما `Rectangle` أو `Square`)، والسلوك سيكون ثابتًا عبر الفئتين. يمكن لبرنامج العميل العمل مع `Rectangle` و `Square` دون أي مشاكل، حيث أن كلاً منهما ينفذ الطريقة `get_area()` بشكل صحيح. لا ترث فئة `Square` من `Rectangle` لأن تلك العلاقة غير مناسبة لمبدأ استبدال ليسكوف. بدلاً من ذلك، كلا من `Rectangle` و `Square` ينفذان الطريقة `get_area()` بشكل صحيح.

Questions:

1. Add ”main call” to test the code after LSP.
2. Analyze the following scenario and identify the violation of the Liskov Substitution Principle (LSP):
   - A *Person* class with *get_salary()* and *assign_task()* methods.
   - A *Student* class is derived from *Person* class.
   - The *Student* class is forced to implement methods that are irrelevant to its functionality, such as *get_salary()* and *assign_task()*.
   - If client code expects a *Person* object and calls these methods, the *Student* class will raise exceptions.
   - Explain how this behavior violates LSP and suggest a solution to fix it.

سؤال:

1. أضف قسم main لفحص وتنفيذ الكود بعد تعديله حسب مبدأ الاستبدال.

2. قم بتحليل السيناريو التالي وتحديد الانتهاك لمبدأ استبدال ليسكوف :(LSP)

- فئة *Person* بها أساليب *get_salary*() و *assign_task*().
- فئة *Student* مشتقة من *Person*.
- فئة *Student* مجبرة على تنفيذ طرق غير ذات صلة بوظائفها، مثل *get_salary*() و *assign_task*().
- إذا كان كود العميل يتوقع كائنًا من نوع *Person* واستدعى هذه الطرق، فإن فئة *Student* سترتفع الاستثناءات.
- اشرح كيف ينتهك هذا السلوك مبدأ استبدال ليسكوف واقترح حلًا لإصلاحه.

4. Interface Segregation Principle (ISP)

The Interface Segregation Principle states that ”clients should not be forced to depend on interfaces they do not use.” In other words, a class should only implement methods it actually needs. This keeps code clean, prevents unnecessary dependencies, and makes it easier to extend or change without side effects.

مبدأ فصل الواجهات:

ينص هذا المبدأ على أنه "يجب ألا يُجبر العملاء على الاعتماد على واجهات لا يستخدمونها". بمعنى آخر، يجب أن يقوم الصنف بتنفيذ الأساليب التي يحتاج إليها فقط. هذا يحافظ على نظافة الكود، ويمنع الاعتمادات غير الضرورية، ويسهل التوسيع أو التغيير دون تأثيرات جانبية.

Example:

Imagine a document printing system with various devices capable of different functionalities. Some devices, like an advanced printer, can print, scan, and fax. However, simpler devices might only print, with no need for scanning or faxing functionality.

نفترض نظام طباعة مستندات مزودًا بأجهزة لها وظائف مختلفة. من بينها: طابعات متقدمة تطبع وتمسح ضوئيا وترسل الفاكس، وطابعات بسيطة تطبع فقط.

Before ISP:

If we create a single *Printer* interface that includes all functions (*print*, *scan*, and *fax*), then even the simplest devices will need to implement unused methods. This violates the ISP because devices that only print are forced to implement the *scan* and *fax* methods, which they don’t use.

إذا أنشأنا واجهة طابعة *Printer* واحدة فيها وظائف (الطباعة والمسح الضوئي والفاكس)، فسنجبر الطابعات البسيطة على تنفيذ أساليب المسح الضوئي والفاكس، التي لا تستخدمها. مما ينتهك مبدأ فصل الواجهات.

```python
class Printer:
    def print_document(self, document):
        pass

    def scan_document(self, document):
        pass

    def fax_document(self, document):
        pass

class BasicPrinter(Printer):
    def print_document(self, document):
        print(f"Printing: {document}")

    def scan_document(self, document):
        # Unnecessary method for BasicPrinter
        raise NotImplementedError("This printer cannot scan.")

    def fax_document(self, document):
        # Unnecessary method for BasicPrinter
        raise NotImplementedError("This printer cannot fax.")


if __name__ == '__main__':
        basic_printer = BasicPrinter()
        basic_printer.print_document("Document2.pdf")
        basic_printer.scan_document("Document2.pdf")
        basic_printer.fax_document("Document2.pdf")
```

In this example, *BasicPrinter* has to implement *scan_document* and *fax_document*, even though it doesn’t support these features. This design is inflexible, with potentially empty or error-prone implementations.

في هذا المثال، يتعين على الطابعة البسيطة BasicPrinter تنفيذ المسح الضوئي *scan_document* وإرسال الفاكس *fax_document* على الرغم من أنها لا تدعم هذه الميزات. هذا التصميم غير مرن، مع احتمال أن يكون الكود فارغاً أو معرّض ا للأخطاء.

After ISP:

To apply the ISP, we can break down the *Printer* interface into smaller, more specific interfaces, each representing a single responsibility: *Printable*, *Scannable*, and *Faxable*. This way, a class only implements the interfaces it needs.

لتطبيق ISP، نقسم واجهة الطابعة *Printer* إلى واجهات صغيرة محددة، كل منها تمثل مسؤولية واحدة: قابل للطباعة *Printable* وقابل للمسح *Scannable* و قابل للإرسال *Faxable*. بهذه الطريقة، تنفذ الفئة الواجهات التي تحتاج إليها فقط.

```python
# Separate Interfaces
class Printable:
    def print_document(self, document):
        pass

class Scannable:
    def scan_document(self, document):
        pass

class Faxable:
    def fax_document(self, document):
        pass

# Implementing Classes
class BasicPrinter(Printable):
    def print_document(self, document):
        print(f"Printing: {document}")

class AdvancedPrinter(Printable, Scannable, Faxable):
    def print_document(self, document):
        print(f"Printing: {document}")

    def scan_document(self, document):
        print(f"Scanning: {document}")

    def fax_document(self, document):
        print(f"Faxing: {document}")
```

This approach ensures that the document printing system can support various device capabilities while keeping the codebase clean, modular, and extensible.

تضمن هذه المنهجية أن يدعم نظام طباعة المستندات قدرات أجهزة مختلفة مع الحفاظ على برنامج نظيف مقسّم قابل للتوسيع.

Questions:

1. Add "main call" to test the code after ISP.
2. The university employs three types of academics:
   - Professor Researcher
   - Full-time Researcher who does not teach
   - Professor Hospital Researcher, whose tasks are: medicine, teaching and research.

   Design the three objects respecting the interface segregation principal.

سؤال:

1. أضف قسم main لفحص وتنفيذ الكود بعد تعديله حسب مبدأ فصل الواجهات.
2. توظف الجامعة ثلاثة أنواع من الأكاديميين:
   - أستاذ جامعي باحث

- باحث متفرغ لا يُدرّس
- أستاذ جامعي باحث استشفائي، مهامه: الطب، التدريس والبحث.

صمم الكائنات الثلاثة مع احترام مبدأ فصل الواجهات.

5. Dependency Inversion Principle (DIP)

High-level modules should not depend on low-level modules. Both should depend on abstractions.
مبدأ عكس التبعية أو الاعتمادية يجب أن لا تعتمد الفئات عالية المستوى على الفئات منخفضة المستوى. يجب أن تعتمد كلتاهما على التجريدات.

Example: Imagine a scenario where a `Notification` class is responsible for sending notifications to users through different channels (like email or SMS). Without following DIP, the `Notification` class may directly depend on specific implementations for each notification method, making it difficult to add new channels (like push notifications) without modifying the existing code.
نفترض في هذا المثال، أنّ الصنف (الفئة) `Notification` يرسل إشعارات إلى المستخدمين عبر قنوات مختلفة (مثل البريد الإلكتروني أو الرسائل النصية). دون اتباع مبدأ عكس الاعتماد (DIP)، قد يعتمد الصنف `Notification` مباشرةً على تنفيذات محددة لكل طريقة إرسال، مما يجعل من الصعب إضافة قنوات جديدة (مثل إشعارات الدفع) دون تعديل الكود الموجود.

Before DIP:
In this example, the `Notification` class depends directly on both `EmailService` and `SMSService`, violating the Dependency Inversion Principle.
في هذا المثال، يعتمد الصنف `Notification` مباشرةً على كل من `EmailService` و`SMSService`، مما ينتهك مبدأ عكس الاعتماد.

```python
class EmailService:
    def send_email(self, message):
        print(f"Sending email: {message}")

class SMSService:
    def send_sms(self, message):
        print(f"Sending SMS: {message}")

class Notification:
    def __init__(self):
        self.email_service = EmailService()
        self.sms_service = SMSService()

    def send(self, message, method):
        if method == "email":
            self.email_service.send_email(message)
        elif method == "sms":
            self.sms_service.send_sms(message)

if __name__ == '__main__':
    my_notifier = Notification()
    my_notifier.send("Salam", "email")
```

After DIP: Now we introduce an abstraction `NotificationService` for different notification types. `Notification` depends on this abstraction, allowing easy addition of new notification services (like push notifications) without modifying `Notification`.
الآن نقدم تجريدًا `NotificationService` لأنواع الإشعارات المختلفة. يعتمد `Notification` على هذا التجريد، مما يسمح بإضافة خدمات إشعارات جديدة بسهولة (مثل إشعارات الدفع) دون تعديل `Notification`.

```python
from abc import ABC, abstractmethod

# Abstract Interface
class NotificationService(ABC):
    @abstractmethod
    def send(self, message):
        pass

# Concrete Implementations
```

```python
class EmailService(NotificationService):
    def send(self, message):
        print(f"Sending email: {message}")

class SMSService(NotificationService):
    def send(self, message):
        print(f"Sending SMS: {message}")

# High-Level Module
class Notification:
    def __init__(self, service: NotificationService):
        self.service = service

    def send(self, message):
        self.service.send(message)
if __name__ == '__main__':
    # Send an email
    email_service = EmailService()
    notification = Notification(email_service)
    notification.send("Hello via Email!")

    # Send an SMS
    sms_service = SMSService()
    notification = Notification(sms_service)
    notification.send("Hello via SMS!")
```

With this design, if you want to add a new notification type (like `PushService`), you can simply implement `NotificationService` in a new class without modifying the `Notification` class. This adheres to DIP by ensuring the `Notification` class depends only on the abstraction `NotificationService` instead of concrete classes.
بهذا التصميم، إذا رغبت في إضافة نوع جديد من الإشعارات (مثل `PushService`)، يمكنك ببساطة تنفيذ `NotificationService` في صنف جديد دون تعديل الصنف `Notification`. يتماشى هذا مع مبدأ عكس الاعتماد (DIP) من خلال ضمان اعتماد الصنف `Notification` فقط على التجريد `NotificationService` بدلاً من الأصناف المحددة.

Questions:

1. Add "Push notification service".
2. Add "main call" to test the code after DIP.
3. **Analyse the given code**: In the current design, the `Application` class is tightly coupled to the `FileLogger` class. If we wanted to change the logging mechanism (e.g., switch from file logging to database logging), we would have to modify the `Application` class. This violates the **Open/Closed Principle** and **Dependency Inversion Principle**.
4. Refactor the code to apply the **Dependency Inversion Principle (DIP)**:
   - Define an abstraction (`ILogger`) for logging.
   - Modify the `Application` class so it depends on the abstraction (`ILogger`), not on concrete logging classes.
   - Implement different logging mechanisms (e.g., `FileLogger`, `DatabaseLogger`) that follow the `ILogger` interface.
   - Ensure that the `Application` class can be used with any logging mechanism without needing to modify the `Application` class.

```python
# Before applying DIP (incorrect design)
class FileLogger:
    def log(self, message):
```

```
        print(f"Logging to file: {message}")

class Application:
    def __init__(self):
        self.logger = FileLogger()

    def start(self):
        self.logger.log("Application started")

# Client code
app = Application()
app.start()
```

سؤال:

1. أضف نوعا جديدا من الإشعارات "Push notfiication service".
2. أضف قسم main لفحص وتنفيذ الكود بعد تعديله حسب مبدأ عكس الاعتمادية.
3. **حلل الكود المعطى**: فئة `Application` مرتبطة ارتباطًا وثيقًا بفئة `FileLogger`. إذا أردنا تغيير آلية التسجيل (على سبيل المثال، التحويل من تسجيل في ملف إلى تسجيل في قاعدة بيانات)، فسنحتاج إلى تعديل فئة `Application`. هذا ينتهك مبدأ **الفتح/الإغلاق ومبدأ عكس الاعتمادية**.
4. إعادة هيكلة الكود لتطبيق **مبدأ عكس الاعتمادية (DIP)**:
   - عرّف تجريد (`ILogger`) للتسجيل.
   - عدّل فئة `Application` بحيث تعتمد على التجريد (`ILogger`) وليس على فئات التسجيل المحددة.
   - نفّذ آليات تسجيل مختلفة (مثل `FileLogger`، `DatabaseLogger`) مشتقة من `ILogger`.
   - التأكد من أنه يمكن استخدام فئة `Application` مع أي آلية تسجيل دون الحاجة إلى تعديل فئة `Application`.

1 Homework

Develop the **Monster** object according to the following requirements:

- **Add health and a health display:** Add `health` to the monster and create a component responsible for displaying its health bar, without making the monster responsible for rendering. → **SRP**
- **Add new monster types:** Add types such as Fast Monster, Flying Monster, and Strong Monster, while allowing them to be added without modifying the monster's core code. → **OCP**
- **Add specialized monster types:** Add different monsters, such as a normal monster, flying monster, and immobile monster, while allowing each type to be used wherever `Monster` is expected without breaking the game logic. → **LSP**
- **Add different monster abilities:** Some monsters move, some shoot, some fly, and some have shields, without forcing every monster to implement abilities it does not need. → **ISP**
- **Change how monsters are affected by attacks:** Some monsters may be affected by bullets, others may be resistant to certain weapons, and others may receive additional damage from specific weapons. The monster should depend on an **abstract damage-calculation mechanism** rather than being directly coupled to a specific weapon or damage method. → **DIP**

واجب منزلي

تطوير كائن **الوحش** وفق المتطلبات التالية:

- **إضافة حياة للوحش ونظام لعرضها** إضافة `health` للوحش، مع إنشاء مكوّن مسؤول عن عرض شريط الحياة، دون تحميل كائن الوحش مسؤولية العرض. **SRP**
- **إضافة أنواع جديدة من الوحوش** مثل وحش سريع، وحش طائر، وحش قوي، مع إمكانية إضافتها دون تعديل الشيفرة الأساسية للوحش. **OCP**
- **إضافة أنواع متخصصة من الوحوش** مثل وحش عادي، وحش طائر، ووحش لا يمكنه الحركة، مع إمكانية استخدام كل نوع مكان `Monster` دون كسر منطق اللعبة. **LSP**
- **إضافة قدرات مختلفة للوحوش** بعض الوحوش تتحرك، بعضها يطلق النار، بعضها يطير، وبعضها يملك درعًا، بحيث لا يُجبر الوحش على امتلاك قدرات لا يحتاجها. **ISP**
- **تغيير طريقة تأثر الوحش بالهجوم** بعض الوحوش تتأثر بالرصاص، وأخرى مقاومة لنوع معين من الأسلحة، وأخرى قد تتلقى ضررًا إضافيًا من أسلحة محددة. يجب أن يعتمد الوحش على **آلية مجردة لحساب الضرر** بدل الارتباط مباشرة بنوع سلاح أو طريقة محددة. **DIP**